CS 645

Security and Privacy in Computer Systems

Lecture 6: Security and Access Control Models

- Policy, models, trust
- Access control models

Last Week

- Application Program Security
  - Buffer overflow
    - Stack-smashing attack
    - NOP sledding
    - Jump to register attack
    - Return-to-libc attack
  - Prevention techniques
    - Educate programmers
    - Use canary
    - Non-executable stack
    - Address-space layout randomization

Announcements

- Project #1 is due next week (Feb 28)

Policy, models, trust

Security Policy

- A **security policy** is a well-defined set of rules that includes the following components:
  - Subjects
  - Objects
  - Actions
  - Permissions
  - Protections
- **Subjects:** the agents who interact with the system
  - specific individuals, or roles or ranks that groups of individuals might hold within an organization.
  - Individuals could be identified by their names or by their job titles, like President, CEO, or CFO.
  - Groups could be defined using terms such as users, administrators, generals, majors, faculty, deans, managers, etc
  - Groups also include outsiders, such as attackers and guests.

Security Policy

- **Objects:** the informational and computational resources that a security policy is designed to protect and manage.
  - Examples: critical documents, files, and databases, and computational resources include servers, workstations, and software.

- **Actions:** the things that subjects may or may not do with respect to the objects.
  - Examples: the reading and writing of documents, updating software on a web server, and accessing the contents of a database.

- **Permissions:** mappings between subjects, actions, and objects, which clearly state what kinds of actions are allowed or disallowed.

- **Protections:** the specific security features or rules that are included in the policy to help achieve particular security goals, such as confidentiality, integrity, availability, or anonymity.

Security Policy

- A security policy places constraints on what actions the subjects in a system can do with respect to the objects in that system, in order to achieve specific security goals

- Regulatory compliance requires organizations such as hospitals, financial institutions, or public corporations to have such policies
  - HIPAA (Health Insurance Portability and Accountability Act)
  - GLBA (Gramm-Leach-Bliley Act)
  - SOX (Sarbanes-Oxley Act)

Security Models

- A **security model** is an abstraction that provides a conceptual language for administrators to specify security policies.
- Typically, security models define hierarchies of access or modification rights that members of an organization can have
  - subjects in an organization can easily be granted specific rights based on the position of these rights in the hierarchy.
- Examples: military classifications of access rights for documents based on concepts like “unclassified,” “confidential,” “secret,” and “top secret.”

U.S. government image in the public domain.

Discretionary Access Control

- **Discretionary access control,** or **DAC,** refers to a scheme where users are given the ability to determine the permissions governing access to their own files.
  - DAC typically features the concept of both users and groups, and allows users to set access-control measures in terms of these categories.
  - In addition, DAC schemes allow users to grant privileges on resources to other users on the same system.

Mandatory Access Control

- **Mandatory access control** is a more restrictive scheme that does not allow users to define permissions on files, regardless of ownership. Instead, security decisions are made by a central policy administrator (system-wide policy).
  - Each security rule consists of a **subject,** which represents the party attempting to gain access, an **object,** referring to the resource being accessed, and a series of permissions that define the extent to which that resource can be accessed.
- **Security-Enhanced Linux (SELinux)** incorporates mandatory access control.

Trust Management

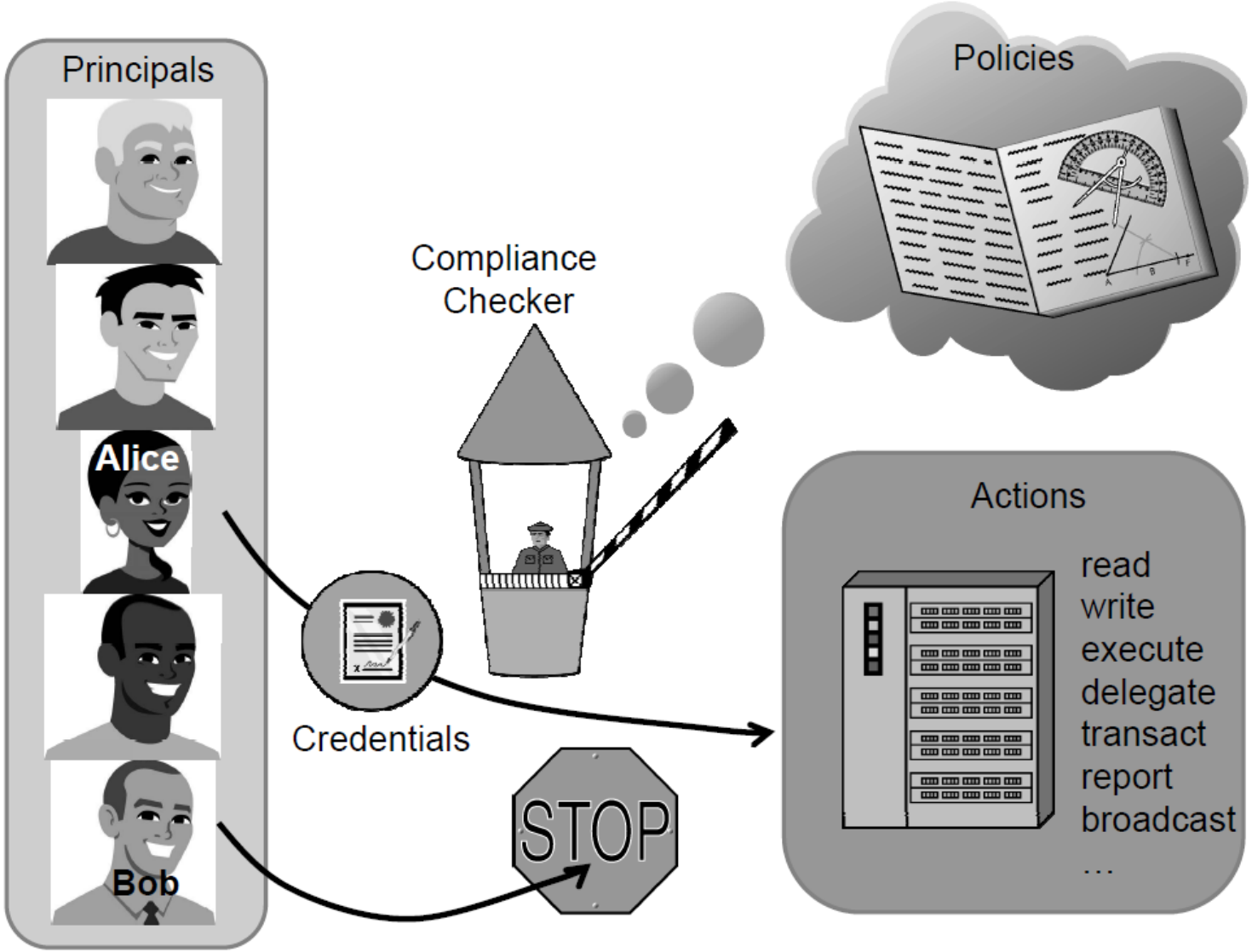
- A **trust management system** is a formal framework that includes:
  - a precise language to specify a security policy, (usually a type of logic or programming language
  - a mechanism for ensuring that the specified policy is enforced.
- A trust management system consists of two main components:
  - a **policy language**
  - a **compliance checker**
- Policy rules are specified in the policy language and are enforced by the compliance checker.

Trust Management Systems

A **trust management system** typically has rules describing:

- **Actions:** operations with security-related consequences on the system
- **Principals:** users, processes, or other entities that can perform actions on the system
- **Policies:** precisely written rules that govern which principals are authorized to perform which actions
- **Credentials:** digitally signed documents that bind principal identities to allowable actions, including the authority to allow principals to delegate authority to other principals.

Trust Management Systems



Trust Management Systems - Keynote

- A policy language proposed in 1999
- ***Applications*** (programs or systems) can use it
- KeyNote issues a ***policy compliance value*** (PCV) in response to a ***request*** by a principal to perform some action
- PCV indicates whether or not the action should be allowed, and the application behaves accordingly
- KeyNote simply interprets whether or not a given action should be permitted according to the provided policies
  - It is up to the application to invoke KeyNote properly and correctly interpret its responses

Trust Management Systems - Keynote

- *"Experience with the KeyNote Trust ManagementSystem: Applications and Future Directions",* by Blaze, Ioannidis, and Keromytis
- https://www.cs.columbia.edu/~angelos/Papers/newkeynote.pdf
  - Network-layer Access Control
  - Distributed firewalls
  - Web access control
  - Micropayments
  - Grid computing

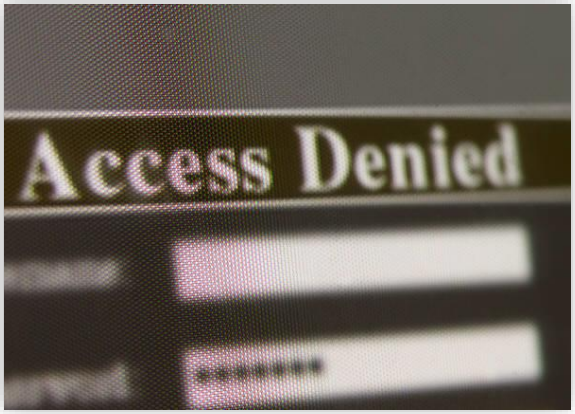
Trust Management Systems - XACML

- A newer policy language proposed in 2009
- XACML stands for ***Extensible Access Control Markup Language***
- Uses XML (Extensible Markup Language) to
  - define security policies
  - describe how these policies should be enforced
- A ***subject*** can perform an action on a ***resource***
- A **policy administration point (PAP)** manages security policies
- Authorizations are issued by a ***policy decision point (PDP)*** (similar to how KeyNote issues PCVs)
- A ***policy enforcement point (PEP)*** requests access on behalf of a subject
  - The PEP behaves in accordance to PDP's responses
- A ***policy information point (PIP)*** provides additional security-related information

Access control models

Access Control Models

- Various models have been developed to formalize mechanisms to protect the confidentiality and integrity of documents stored in a computer system.
  - The Bell-La Padula (BLP) model
  - The Biba model
  - The Low-Watermark model
  - The Clark-Wilson model
  - The Chinese Wall model (The Brewer and Nash model)
  - Role-based (RBAC)
  - History-based

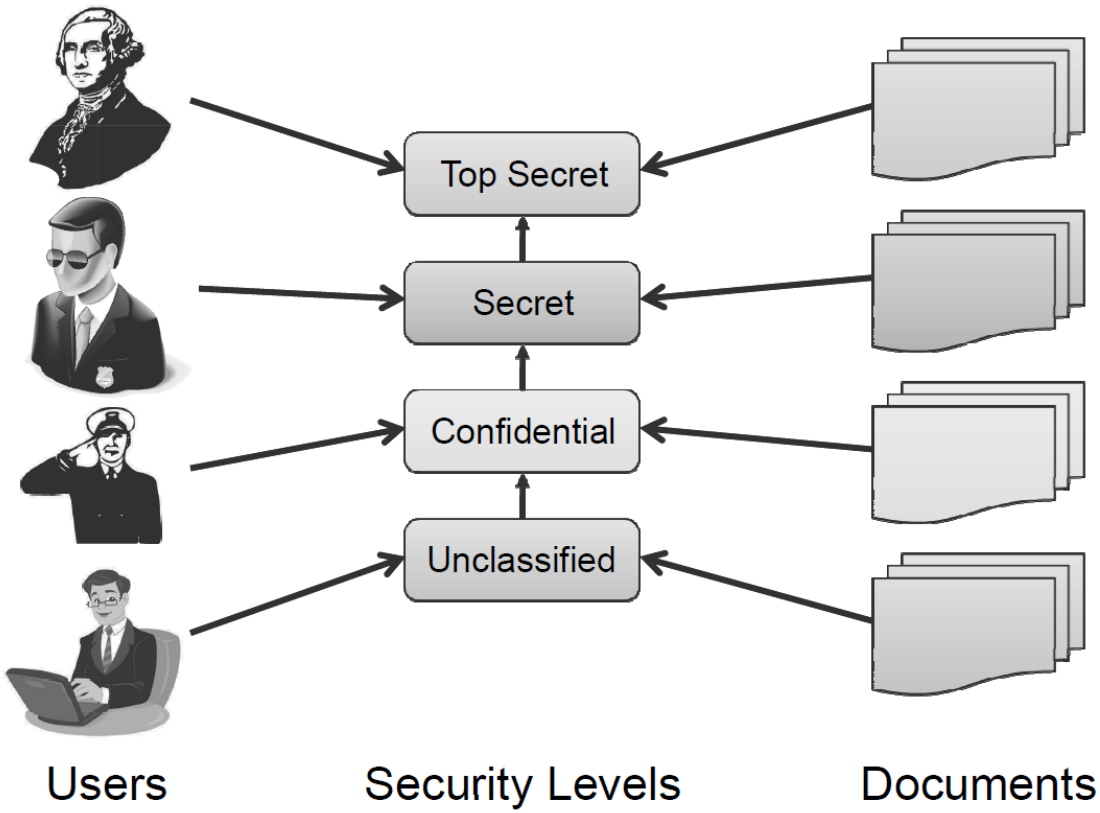


The Bell-La Padula Model

- The **Bell-La Padula (BLP) model** is a classic mandatory access-control model for protecting ***confidentiality***.
- Derived from the military **multilevel security paradigm,** which has been traditionally used in military organizations for document classification and personnel clearance.
- When BLP model was introduced, the Air Force was concerned with security in time-sharing systems: OS bugs, accidental misuse
- The objective: formally show that a computer system can securely process classified information

The Bell-La Padula Model

- The BLP model has a strict, linear ordering on the security of levels of documents
  - each document has a specific security level in this ordering
  - each user is assigned a strict level of access that allows them to view all documents with the corresponding level of security or below.



The Bell-La Padula Model – multilevel security

- There are several security levels:
  - Military systems: unclassified ≤ confidential ≤ secret ≤ top secret
  - Commercial systems: public ≤ sensitive ≤ proprietary ≤ restricted
- Each document is classified at one of the security levels
- Each user obtains “clearance” at one of the security levels
- A document at a certain level can be accessed only by users with the same or higher clearance level

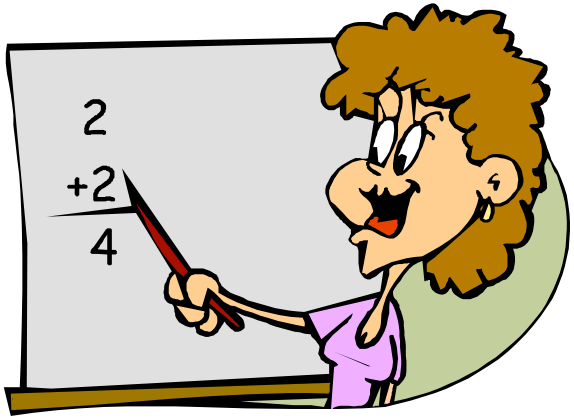
The BLP model

- A computer system is modeled as a state-transition system
  - A system is secure if every state that it can reach satisfies certain properties
  - Each subject S is given a security level L(S), and each object O is given a security level L(O)
  - Access attributes: read, write, copy, append, owner, control
  - Requests for access to objects by subjects
  - Decisions: yes, no, error
  - In each state, there are subjects, objects, an access matrix, and the current access information
  - There are state transition rules describing how a system can go from one state to another

The BLP model

- A state is secure if it satisfies:
  - Simple Security Condition (no read up):
    - S can read O iff L(S) ≥ L(O)
  - The Star Property (no write down)
    - S can write O iff L(S) ≤ L(O)
  - Discretionary-security property
    - a subject (or role) may grant to another subject access to an object, constrained by the Mandatory Access Control rules
- A system is secure if and only if every reachable state is secure.

Begin Math

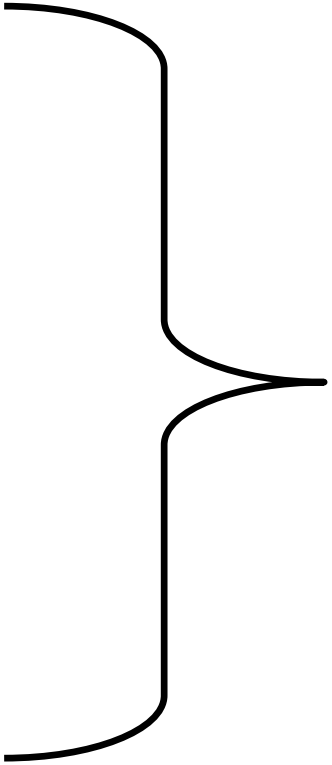


Partially Ordered Set

- A Set S with relation $\leq$ (written $(S, \leq)$) is called a ***partially ordered set*** if $\leq$ satisfies
  - Reflexivity
    - For all a in S, $a \leq a$
  - Antisymmetry
    - If $a \leq b$ and $b \leq a$ then $a = b$
  - Transitivity
    - For all a, b, c, $a \leq b$ and $b \leq c$ implies $a \leq c$

- NOTE: "$a \leq b$" reads "a precedes b"

End Math



How the BLP Model Works

- The security levels in BLP form a partial order, $\leq$.
- Each object, x, is assigned to a security level, L(x). Similarly, each user, u, is assigned to a security level, L(u).
- Access to objects by users is controlled by the following two rules (we ignore the third rule for simplicity):
  - **Simple security property:** A user u can read an object x if and only if:

    $$L(x) \leq L(u).$$

  - ***-property:** A user u can write (create, edit, or append to) an object x if and only if:

    $$L(u) \leq L(x).$$

- The simple security property is also called the "no read up" rule, as it prevents users from viewing objects with security levels higher than their own.
- The *-property is also called the "no write down" rule. It is meant to prevent propagation of information to users with a lower security level.

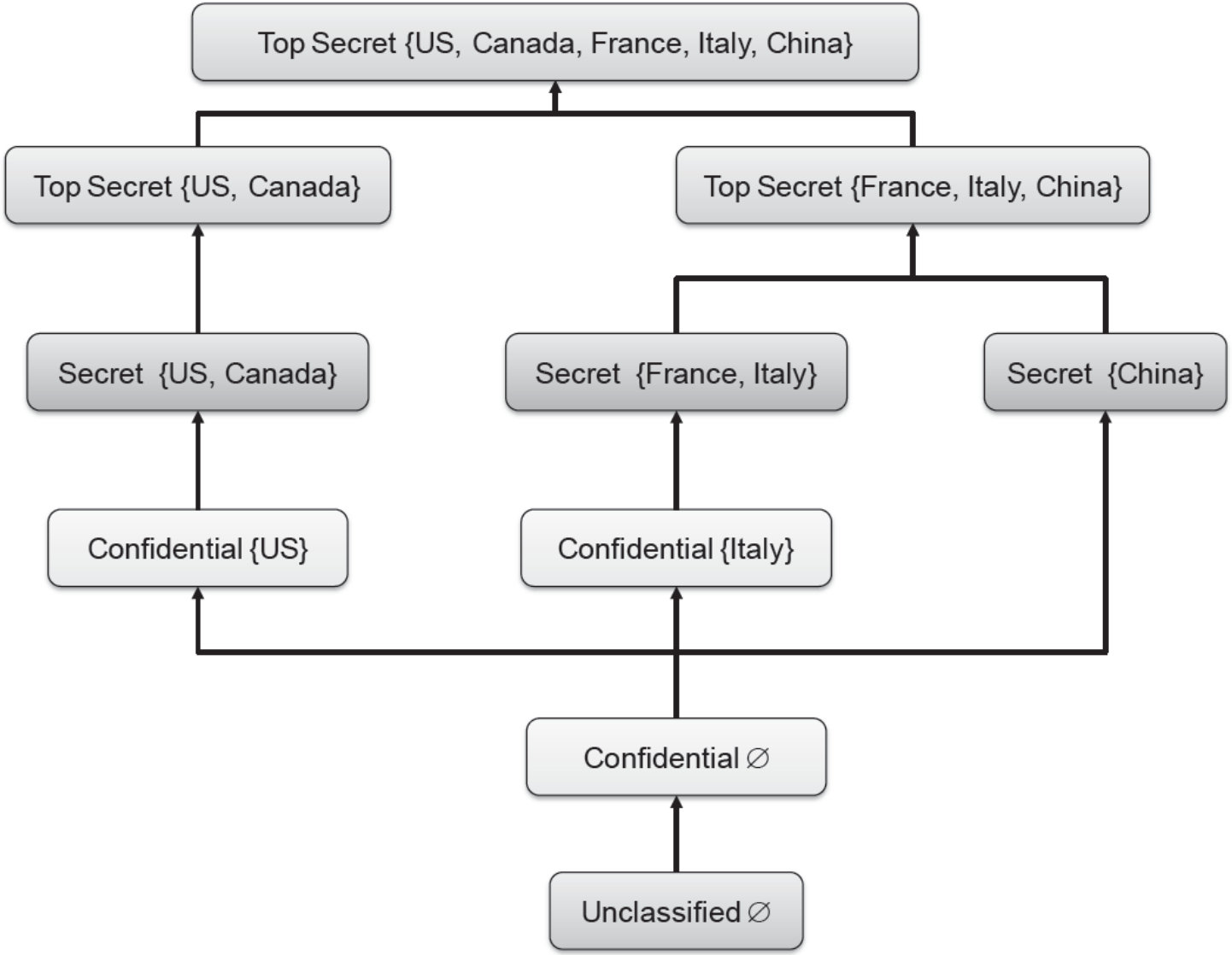
How the BLP Model Works

- Users with different security levels can have only one-way communication
  - If $L(u) \leq L(v)$, then u can send a message to v (u writes up the message and v reads down the message)
  - The opposite is not possible (this is a limitation!)
- To overcome this problem, we can change the meaning of L(u) to be the ***maximum security level*** that u can have and allow u to assume a ***current security level***, C(u), such that $C(u) \leq L(u)$.
  - Now, if If $L(u) \leq L(v)$, user v can assume current security level $C(v) = L(u)$ to have a two-way communication with user u

How the BLP Model Works

- In practical application of the BLP model, a security level L(x) consists of a pair (b(x), S(x)), where
  - b(x) is a level from a set of ***basic levels*** B (the levels in B have a partial order defined on them)
  - S(x) is a set of ***categories*** from a collection of ***categories*** S (called ***compartments***)
- The level of x precedes the level of y if and only if the basic level of x is less than the basic level of y, and the subset of categories of x is contained in the subset of categories of y
  - (b(x), S(x)) $\leq$ (b(y), S(y)) if and only if b(x) $\leq$ b(y) and S(x) is included in S(y)

Defining Security Levels Using Categories



Issues with BLP

- Deals only with confidentiality,
  - does not address integrity
- Does not deal with information flow through covert channels
- The approach of defining a secure system to be one in which every reachable state is secure is flawed
  - in a system that is secure according to BLP, a subject can read high, change current level to low, then write low .
  - to address this problem, need to require
    - subjects cannot change current levels
    - or ensure that a subject “forgets” everything when changing levels

The Biba Model

- The **Biba model** has a similar structure to the BLP model, but it addresses ***integrity*** rather than confidentiality.
  - Goal is to prevent unauthorized modification of data
  - The data can be visible at some or all security levels, but should be modified in a controlled way

- Objects and users are assigned **integrity levels** that form a partial order, similar to the BLP model.
  - Higher levels are more reliable/trustworthy

The Biba Model

- The integrity levels in the Biba model indicate degrees of trustworthiness for objects and users, rather than levels for determining confidentiality.
  - For example, a file stored on a machine in a closely monitored data center would be assigned a higher integrity level than a file stored on a laptop.
  - In general, a data-center computer is less likely to be compromised than a random laptop computer. Likewise, when it comes to users, a senior employee with years of experience would have a higher integrity level than an intern.
  - Higher levels are more reliable/trustworthy
- Integrity levels are different from security levels in confidentiality models
  - Non-sensitive data may have high integrity
  - Highly sensitive data may have low integrity

The Biba Model Rules

- The access-control rules for Biba are the reverse of those for BLP.
  - Biba does not allow reading from lower levels and writing to upper levels.
- If we let L(u) denote the integrity level of a user u and L(x) denote the integrity level for an object x, we have the following rules in the Biba model:
  - A user u can read an object x if and only if

    $L(u) \leq L(x)$. (no read down)
  - A user u can write (create, edit or append to) an object x if and only if

    $L(x) \leq L(u)$. (no write up)
- Thus, the Biba rules express the principle that information can only flow down, going from higher integrity levels to lower integrity levels.

The Low-Watermark Model

- The **low-watermark model** is an extension to the Biba model that relaxes the “no read down” restriction, but is otherwise similar to the Biba model.
- In other words, users with higher integrity levels can read objects with lower integrity levels.
- After such a reading, the user performing the reading is demoted such that his integrity level matches that of the read object.

The Clark-Wilson Model

- Rather than dealing with document confidentiality and/or integrity, the **Clark-Wilson (CW)** model deals with systems that perform transactions.
- It describes mechanisms for assuring that the integrity of such a system is preserved across the execution of a transaction. Key components of the CW model include the following:
  - **Integrity constraints** that express relationships among objects that must be satisfied for the system state to be valid.
    - Example: the relationship stating that the final balance of a bank account after a withdrawal transaction must be equal to the initial balance minus the amount withdrawn.
  - **Certification methods** that verify that transactions meet given integrity constraints.
    - Once the program for a transaction is certified, the integrity constraints do not need to be verified at each execution of the transaction.
  - **Separation of duty rules** that prevent a user that executes transaction from certifying it.
    - Each transaction is assigned disjoint sets of users that can certify and execute it, respectively.

The Chinese Wall Model

- The **Brewer and Nash model,** commonly referred to as the **Chinese wall model,** is designed for use in the commercial sector to eliminate the possibility of conflicts of interest.
- To achieve this, the model groups resources into “conflict of interest classes”.
- The model enforces the restriction that each user can only access one resource from each conflict of interest class.
  - In the financial world, such a model might be used, for instance, to prevent market analysts from receiving insider information from one company and using that information to provide advice to that company’s competitor.
- Such a policy might be implemented on computer systems to regulate users’ access to sensitive or proprietary data.

Role-Based Access Control

- The **role-based access control (RBAC)** model can be viewed as an evolution of the notion of group-based permissions in file systems.
- An RBAC system is defined with respect to:
  - an organization (such as a company)
  - a set of resources (such as documents, print services, and network services)
  - a set of users (such as employees, suppliers, and customers)

U.S. Navy image in the public domain.

RBAC components

- A **user** is an entity that wishes to access resources of the organization to perform a task.
  - Usually, users are actual human users, but a user can also be a machine or an application.
- A **role** is defined as a collection of users with similar functions and responsibilities in the organization.
  - Examples of roles in a university may include “student,” “alum,” “faculty,” “dean,” “staff,” and “contractor.” In general, a user may have multiple roles.
  - Roles and their functions are often specified in the written documents of the organization.
  - The assignment of users to roles follows resolutions by the organization, such as employment actions (e.g., hiring and resignation) and academic actions (e.g., admission and graduation).

RBAC components

- A **permission** describes an allowed method of access to a resource**.**
  - More specifically, a permission consists of an operation performed on an object, such as “read a file” or “open a network connection.”
  - Each role has an associated set of permissions.

- A **session** consists of the activation of a subset of the roles of a user for the purpose of performing a certain task.
  - For example, a laptop user may create a session with the administrator role to install a new program.
  - Sessions support the principle of least privilege.

Why use roles?

- Roles are a useful level of abstraction
- Organizations operate based on roles
- There are fewer relationships to manage
  - for m users and n objects, possibly decrease from m*n to m+n
  - There are usually more users than roles and more objects than roles
- Roles are likely to be more stable than the set of users and the set of objects

Roles vs. Groups

- How are roles different from groups?
  - Groups refer to sets of users; roles refer to sets of users with associated permissions
  - Roles can be activated and deactivated; groups cannot
  - Permissions of a role can be easily enumerated; not true of a group

Hierarchical RBAC

- In the role-based access control model, roles can be structured in a hierarchy similar to an organization chart.
- More formally, we define a partial order among roles by saying that a **role R1 inherits role R2,** which is denoted

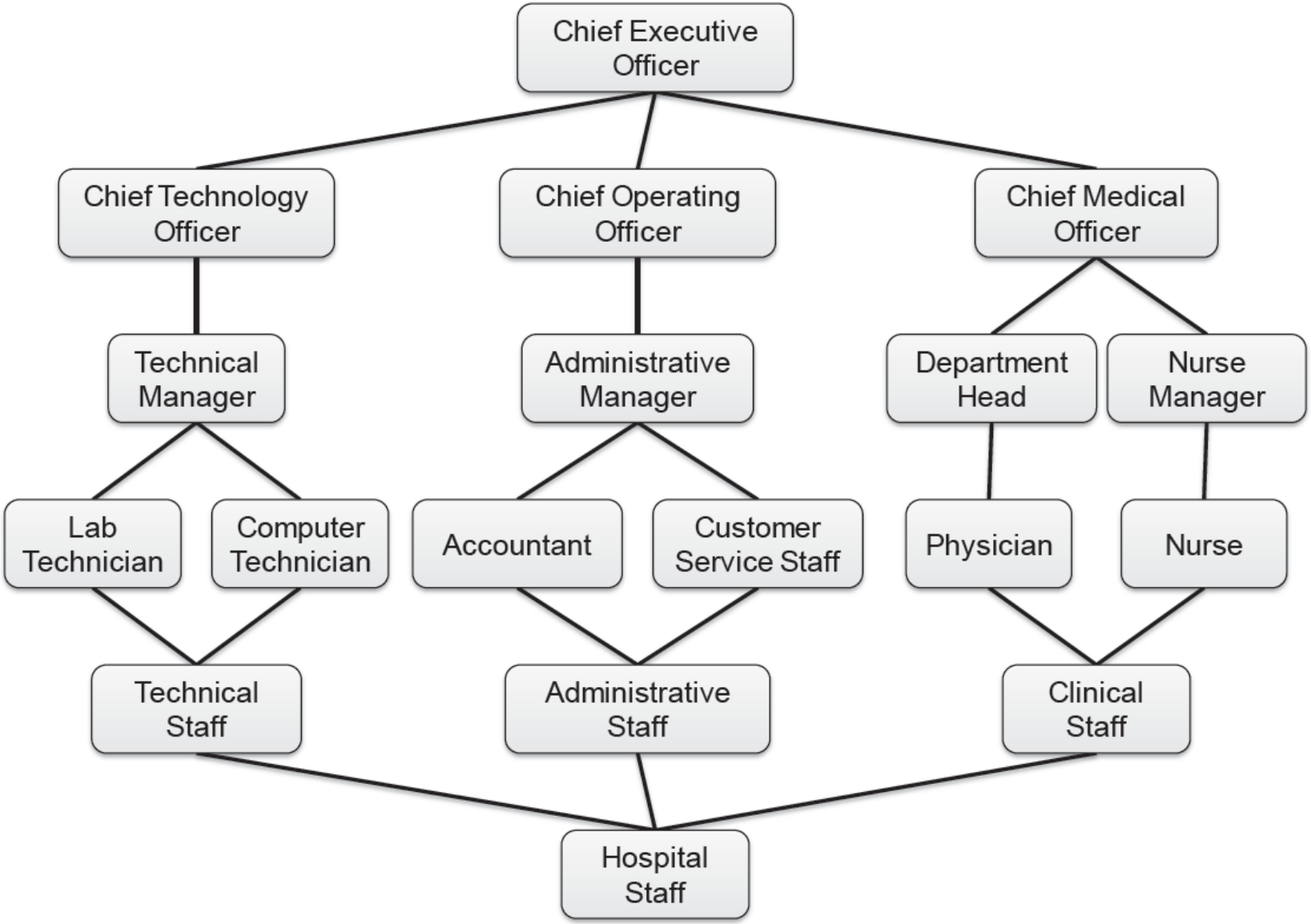
  $$R1 \geq R2,$$

  if R1 includes all permissions of R2 and R2 includes all users of R1.
- When $R1 \geq R2$, we also say that role R1 is **senior** to role R2 and that role R2 is **junior** to role R1.
  - For example, in a company, the role “manager” inherits the role “employee” and the role “vice president” inherits the role “manager.”
  - Also, in a university, the roles “undergraduate student” and “graduate student” inherit the role “student.”

Hierarchical RBAC

- Inheritance simplifies the management of permissions and users associated with roles
  - When a permission is added to a role, the system propagates this permission addition to senior roles
    - Example: adding permission of viewing student grades to the role “professor”, automatically adds this permission also to the role “dean“
  - When a user is added to a role, the system propagates this user addition to all junior roles
    - Example: adding user Mike to the role “professor”, automatically adds Mike to the role “employee”

Visualizing Role Hierarchy



Constrained RBAC

- RBAC provides support for the principle of separation of privileges by defining ***constraints***
  - A constraint prevents users from having incompatible roles that create conflicts of interest
- Simplest form of constraint is a pair of roles (R1, R2)
  - No user can be assigned to both roles R1 and R2
  - Example: in a company, no employee should have both the roles of buyer (who proposes purchases of goods) and controller (who reviews and approves purchase orders)

Constrained RBAC

- A more general constraint is a ***separation of duty*** relation, defined by a pair (S,k), where S is a subset of roles and k is an integer
  - No user can have k or more roles from S
- Separation of duties can have two different meanings:
  - ***Static***: the constraint holds for the ***assignment*** of users to roles (no user can be ***assigned*** to k or more roles from S)
  - ***Dynamic***: the constraint holds for the ***activation*** of users to roles (no user can be have k or more roles in S ***activated*** in a session)

Constrained RBAC

- Dynamic separation of duties is more flexible
  - It allows users to have different roles from incompatible sets in different sessions, so long as the sessions don't overlap in time
  - Example:
    - Mary is the head of the research division in a company
    - In one session, Mary activates the role "supervisor" to approve a travel expense report from an employee of her division
    - In another session, Mary activates the role "traveler" to submit her own travel expense report
    - ({supervisor, traveler}, 2) – this dynamic separation of duties relation ensures that Mary cannot approve her own travel expense report

RBAC in Use

- Products that use RBAC
  - database management systems (e.g., Oracle)
  - enterprise security management (e.g., IBM Tivoli Identity Manager)
  - operating systems (e.g., Solaris OS, AIX)
- RBAC economic impact study (2002)
  - was conducted by the Research Triangle Institute (RTI) based on interviews with software developers and companies that use RBAC
  - it estimated by 2006 30–50% of employees in service sector would be managed by RBAC systems (10–25% for non-service sectors)
  - it conservatively estimated the economic benefits of this degree of penetration through 2006 to be $671 million

The NIST RBAC standard

- In 2001 RBAC was proposed to become a NIST standard
- It was adopted as ANSI (American National Standards Institute) standard in 2004

RBAC Today

- The ANSI standard has been criticized by Li et al. (2007)
  - there are many errors
  - there are other limitations and design flaws
  - the publication proposes several changes to the standard
- RBAC is still an active area of research
  - many extensions exist (temporal, geo-spatial, privacy-aware)
  - administration of RBAC
  - constraints, workflow, role engineering, . . .

History-based Access Control

- Access control by history determines whether to grant a subject access to an object based on the subject's history of accesses
  - each access by a subject is logged
- Example 1: limited number of accesses
  - each subject can access an object at most n times
- Example 2: statistical databases
  - allow queries for general statistics, but not individual values
  - valid queries are statistics on 20+ individuals
    - retrieve total salary of all deans
    - retrieve average salary of all CS professors

- Drawback: the need to remember all past queries

What to read?

- Chapters 9.1, 9.2 from the Textbook